General Concepts

General Terms

- **Bandwidth:** max data rate of a link (bits/sec).
- **Goodput:** proportion of successful slots (transmission without collision).
- **Throughput:** proportion of slots with a transmission (success or not).
- **Propagation time:** time for a signal to travel between two nodes.

Mathematical Foundations

Probability

- **Conditional:** $P(A \mid B) = \frac{P(A \cap B)}{P(B)}$, $P(B) > 0$
- **Bayes:** $P(A \mid B) = \frac{P(B|A)P(A)}{P(B)}$
- **Total Prob. Law:** $P(B) = \sum_i P(B \mid A_i)P(A_i)$ (partition $\{A_i\}$)
- **Expected Value:** $\mathbb{E}[X] = \sum_x xP(X = x)$
- **Memoryless:** $P(X > s + t \mid X > t) = P(X > s)$ (Geo/Exp).

Dist.	PMF	Notation	$\mathbb{E}[X]$	$\mathrm{Var}(X)$
Bernoulli	$P(X = 1) = p$	$\mathrm{Ber}(p)$	p	$p(1-p)$
Binomial	$\binom{n}{k} p^k (1-p)^{n-k}$	$\mathrm{Bin}(n, p)$	np	$np(1-p)$
Geometric	$(1-p)^{k-1}p$	$\mathrm{Geo}(p)$	$\frac{1}{p}$	$\frac{1-p}{p^2}$
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	$\mathrm{Pois}(\lambda)$	λ	λ

- **Poisson Process:** events occur at rate λ; inter-arrival times $\sim \mathrm{Exp}(\lambda)$.
- In a Poisson process with rate λ, the probability of k events in time T is $P_k(t = T) = \frac{(\lambda T)^k}{k!} e^{-\lambda T}$.

Computer networks vs Distributed systems

Circuit and Packet Switching

Timing

Multiplexing

The OSI Model (5-Layer model)

- **Physical:** Move bits on a link (signals, timing, encoding).
- **Link:** Frames on local network, MAC addressing, error detection.
- **Network:** Packet delivery across networks, routing (IP).
- **Transport:** End-to-end process communication, reliability/flow control (TCP/UDP).
- **Application:** End-user services/protocols (DNS, DHCP, HTTP, FTP etc.).

Link Layer & Local Area Networks (LAN)

Shared Medium Intuition

- Nodes broadcast on same channel; only one can transmit at a time.
- Simultaneous transmit $\Rightarrow$ collision $\Rightarrow$ data lost.
- Sender uses full link bandwidth while transmitting.

Definitions

- **Bandwidth:** max data rate of channel (B).
- **Throughput:** fraction of bandwidth used to send traffic.
- **Goodput:** fraction of bandwidth used for successful data.
- $\eta = \frac{\text{avg effective bandwidth}}{\text{total bandwidth}} = \frac{\text{avg effective time}}{\text{total time}}$, $0 \le \eta \le 1$.

Multiple Access Domains

- **Multiple Access Domain:** set of nodes sharing a common channel.
- **Channel Partitioning:** divide channel into smaller "pieces" (time/freq/code).
- **Random Access:** transmit at will; handle collisions (ALOHA, CSMA).
- **Taking Turns:** nodes take turns to transmit (polling/token passing).

ALOHA protocol

- Random access on shared medium; collisions if overlap. Frame time T (size L, rate B): $T = L/B$.
- **Binomial model:** N users, each transmits in a slot w.p. p.
- **Poisson model:** aggregate attempts are Poisson with rate G (attempts/slot).
- **Slotted ALOHA (align to slots):**
  - Success if exactly one tx in a slot.
  - Binomial: $P(\text{succ}) = Np(1-p)^{N-1}$, max at $p = 1/N$ gives $S_{\max} \approx 1/e$.
  - Poisson: $S = Ge^{-G}$, $S_{\max} = 1/e$ at $G = 1$.
- **Pure ALOHA (no slots):**
  - Vulnerable period $2T$.
  - Binomial (approx): $P(\text{succ}) \approx Np(1-p)^{2(N-1)}$, max at $p \approx 1/(2N)$ gives $S_{\max} \approx 1/(2e)$.
  - Poisson: $S = Ge^{-2G}$, $S_{\max} = 1/(2e)$ at $G = 1/2$.

CSMA (Carrier Sense Multiple Access)

- **Basic Principle:** Listen (carrier sense) before transmitting.
- **Persistence Strategies:**
  - **1-persistent:** If idle, transmit immediately. If busy, wait until idle and transmit immediately (high collision probability).
  - **Non-persistent:** If idle, transmit. If busy, wait a random time (backoff) and check again.
  - **p-persistent:** If idle, transmit with probability p, else wait for next slot (compromise).
- **Propagation delay** τ (T_{prop}): Time for signal to travel between the two farthest nodes.

CSMA/CD (Collision Detection) - Wired (Ethernet)

- **Mechanism:** Listen while transmitting. If collision detected (voltage spike), abort transmission, send **jam signal**, then backoff.
- **Condition for Detection:** Transmission time must be long enough to detect a collision from the farthest node and receive the collision signal back.

$$T_{\text{trans}} \ge 2 \cdot T_{\text{prop}}$$

(This determines the minimum packet size).
- **Example (minimum frame size):** distance $= 8\,\text{km}$, propagation speed $= 4 \times 10^7\,\text{m/s}$, rate $= 5\,\text{Mbps}$. Then

$$T_{\text{prop}} = \frac{8 \times 10^3}{4 \times 10^7} = 2 \times 10^{-4}\,\text{s} = 200\,\mu s \Rightarrow 2T_{\text{prop}} = 400\,\mu s$$

$$T_{\text{trans}} = \frac{S}{5 \times 10^6} \Rightarrow S = (5 \times 10^6)(4 \times 10^{-4}) = 2000\,\text{bytes}.$$

- **Binary Exponential Backoff:**
  - After m-th collision, choose k uniformly from $\{0, \ldots, 2^m - 1\}$.
  - Wait $k \cdot$ slot time (where slot time ≈ 512 bit times in Ethernet).
- **Efficiency (η):** As $N \to \infty$:

$$\eta = \frac{1}{1 + 2 \cdot \frac{T_{\text{prop}}}{T_{\text{trans}}} \cdot (e - 1)}$$

CSMA/CA (Collision Avoidance) - Wireless (WiFi/802.11)

- **Reasoning:** Cannot detect collisions reliably (weak received signal vs strong transmitted signal), so aim to avoid them.
- **Protocol:**
  - If channel idle for **DIFS**, transmit entire frame.
  - If busy, choose random backoff. **Timer counts down only when channel is idle** (freezes when busy).
  - Receiver sends **ACK** after **SIFS** if frame received successfully (no ACK = collision).
- **Virtual Carrier Sensing (RTS/CTS):** Prevents "Hidden Terminal" problem.
  - Sender transmits **RTS** (Request to Send).
  - Receiver replies **CTS** (Clear to Send).
  - Neighbors hear RTS/CTS and defer transmission (reserve the channel).

Single (Logical) Link in practice

MAC Addresses

- Link-layer address unique per NIC (Network Interface Card); assigned by IEEE OUI.
- Used to deliver frames on same LAN; flat (non-hierarchical), portable across LANs.
- Usually burned-in, but can be software-set.

Ethernet

- Dominant wired LAN; many speed generations; cheap NICs.
- **CSMA/CD (classic shared medium):**
  1. Sense channel; if idle transmit, else wait.
  2. If collision detected while tx, abort and send jam.
  3. **Binary exponential backoff:** after m-th collision pick $k \in \{0, \ldots, 2^m - 1\}$, wait $k \cdot 512$ bit-times, retry.

Interconnecting broadcast domains

- **Switch:**
  - link-layer device that breaks broadcast domains; stores/forwards frames using MAC table.
  - Transparent to hosts; self-learning (no manual config), entries age out (TTL).
  - Each link is its own collision domain; enables simultaneous transmissions.
  - Forwarding: if dest known on incoming port $\Rightarrow$ drop; else forward to port; if unknown $\Rightarrow$ flood.
  - **Switch table:** (MAC, port, TTL). Learn from source MAC+ingress port on each frame.
- **Loops:** cause broadcast storms, multiple frame copies, MAC table instability.
- **Spanning Tree Protocol (STP):** prevent L2 loops by blocking redundant links; converges to a loop-free spanning tree.
  - **BPDUs (Hello messages):** (RID, RootCost, SID, PortID) advertised periodically; switches keep the *best* tuple.
  - **Root election:** lowest RID wins (all switches start assuming they are root).
  - **Root port (per non-root switch):** port with lowest RootCost to root; ties: lower upstream SID, then lower PortID.
  - **Designated port (per LAN segment):** switch with lowest RootCost on that segment forwards; ties: lower SID, then lower PortID.
  - **Forwarding:** root ports + designated ports forward; all others block (no data forwarding, still send BPDUs).

Errors Detection

- Goal: detect (and sometimes correct) bit errors introduced by noise on the link.
- **Repetition code (3-repetition):** send each bit 3 times; bit is chosen by majority vote.
- **1D parity:** add one parity bit so total 1s is even/odd; detects any odd number of bit errors, cannot locate/correct.
- **2D parity:** arrange bits in a matrix with row+column parity; detects many multi-bit errors and can correct a single-bit error (intersection).
- **CRC:** TODO **Consider removing this if not in Exams** treat bits as polynomial over $GF(2)$; append r-bit remainder so frame is divisible by generator $G(x)$.
- CRC detects all single-bit errors, all burst errors of length $\le r$, and most longer bursts (prob. of miss $\approx 2^{-r}$).

Code	Overhead	Ability to correct	Ability to detect
Repetition (3 times)	2	1	2
1D Parity	$\frac{1}{N}$	0	1
2D parity	$\frac{2N+1}{N^2}$	1	3

"Layer 2.5" - ARP

ARP is generally considered to be part of Layer 2 (Data Link Layer), but it works directly to support Layer 3 (Network Layer).

- Why it's Layer 2: ARP messages are encapsulated directly into Ethernet frames. They do not have an IP header (Layer 3) and they cannot be routed outside of your local network (LAN).
- Why it feels like Layer 3: Its entire purpose is to handle Layer 3 information (IP addresses). It is the mechanism that allows Layer 3 to actually function on physical hardware.

Address Resolution Protocol (ARP)

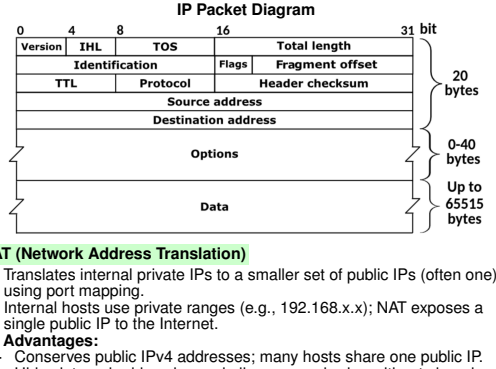
- Translates between MAC addresses and IP addresses on a LAN.
- Needed because Layer 3 provides the IP, but frames on Layer 2 need a destination MAC.
- To send outside the LAN, the packet uses the IP of the destination but the MAC of the next-hop router; routers then forward using their own L2 on each hop.
- Hosts use DHCP and routing to know whether the destination IP is on the same LAN or remote.
- Each node maintains an **ARP cache** (IP $\to$ MAC table).
- If the mapping is missing, send an **ARP request** (broadcast in the local broadcast domain).
- The target replies with an **ARP response** containing its MAC, which is stored in the cache.
- ARP messages stay within a single broadcast domain (not routed).

Network Layer (IP & Routing)

Introduction to IP addressing

- IPv4 address = 32 bits (a.b.c.d), identifies interface (host/router can have multiple interfaces).
- Subnet vs host split via CIDR: a.b.c.d/x where x = prefix length (subnet mask).
- Example: 192.168.1.178/24 $\Rightarrow$ subnet 192.168.1, host 178.
- Hierarchical addressing enables routing scalability; ISPs advertise prefixes.
- **Router:** network-layer device that connects networks and forwards IP packets based on destination prefixes (routing table), decoupling L2 domains.
- **Router + ARP/L2:** for each hop, the router uses ARP on the outgoing interface to learn the next-hop MAC, rewrites the L2 header, and forwards the same IP packet (new L2 header per link).
- Forwarding uses **longest prefix match**.
- **Address discovery protocols:**

IP Packet Diagram



NAT (Network Address Translation)

- Translates internal private IPs to a smaller set of public IPs (often one) using port mapping.
- Internal hosts use private ranges (e.g., 192.168.x.x); NAT exposes a single public IP to the Internet.
- **Advantages:**
  - Conserves public IPv4 addresses; many hosts share one public IP.
  - Hides internal addressing and allows renumbering without changing external addresses.
  - Basic inbound shielding (unsolicited inbound blocked unless mapped).
- **How it works:**
  - NAT maintains a translation table mapping (private IP, private port) $\leftrightarrow$ (public IP, public port).
  - For outbound packets, NAT rewrites source IP:port and records the mapping.
  - For inbound packets, NAT looks up the destination port and rewrites to the internal host; if no mapping, it drops.
- **Why it doesn't fully solve IPv4 shortage:**
  - Breaks end-to-end connectivity; inbound connections need port forwarding/ALGs.
  - Adds per-packet processing and state; can increase latency and complexity.
  - Scalability: still limited by public IPs for NAT gateways and port space.

Topology

- Network graph structure (how nodes/links are connected).
- **Diameter:** max shortest-path distance between any two nodes (in hops/links).
- **Nodal degree:** number of links incident to a node.
- **Bisection bandwidth:** min total bandwidth across any split of nodes into two equal halves.
- **Edge expansion (intuition):** connectivity of small sets; higher expansion $\Rightarrow$ more robust.

Topology (N nodes)	Diameter	Degree	Bisection
Line	$N - 1$	2 (ends 1)	1
Ring	$N/2$	2	2
Balanced Tree	$2 \log_2 N$	3 (root 2, leaves 1)	1
Hypercube ($N = 2^k$)	$\log_2 N$	$\log_2 N$	$N/2$
Complete Bipartite	2	$N/2$	$N^2/8$

Intradomain Routing

- Goal: choose a path for a packet through the network to reach the destination.
- **Shortest path routing:** pick the minimum-cost path (weights represent costs).
- **DV (Distance Vector):** each node knows only its neighbors and their distances (not full topology). Each node keeps a vector of distances to all destinations (anycast).

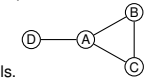
Distance Vector (Bellman-Ford)

For node s:

$$d_s(v) = \begin{cases} c(s, v), & v \in \Gamma(s) \\ 0, & v = s \\ \infty, & \text{else} \end{cases}$$

Iterate:

- Update with neighbors: $d_s(v) = \min_{x \in \Gamma(s)} \{c(s, x) + d_x(v)\}$.
- If any entry changes, propagate updated vector.
- **Count-to-infinity:** when a path fails, nodes can keep increasing distances by looping updates.
- **Poisoned reverse:** if route from X to Z goes via Y, then Y advertises Z to X with ∞ to prevent two-node loops (still fails for loops ≥ 3).
  - Example (3-node loop): equal link costs; if link $A-D$ fails (weight ∞),



  poisoned reverse still fails.
- **RIP (Routing Information Protocol):**
  - Distance-vector protocol using hop count as metric (max 15 hops).
  - Sends full distance vector every 30 seconds.
  - If no update for 180 seconds, route is considered unreachable (infinite).
- **ARPANET:** early distance-vector-like algorithm; suffered from slow convergence and required careful update control to avoid instability; not used today (historical).

Link State (Dijkstra)

- Global knowledge: each node learns full topology and link costs; changes are flooded (multicast/LSA).
- Dijkstra from source s:

Initialize:

$$d_s(v) = \begin{cases} c(s, v), & v \in \Gamma(s) \\ 0, & v = s \\ \infty, & \text{else} \end{cases} \qquad N' = \{s\}$$

Repeat until $N' = N$:

1. Choose $w \notin N'$ with minimal $d_s(w)$.
2. Add w to N'.
3. For each $v \in \Gamma(w) \setminus N'$, update $d_s(v) = \min\{d_s(v), d_s(w) + c(w, v)\}$.

- Once the table is built, forward using it (multicast routing supported).
- **OSPF (Open Shortest Path First):**
  - Link-state protocol; supports multiple metrics/weights, multiple paths, hierarchical areas.
  - Link costs/weights are configured by the network admin.
  - LSAs are flooded (multicast) on change; periodic refresh as well.
  - Each router builds full topology and runs Dijkstra locally.

Comparison of LS vs DV

Aspect	LS (Link State)	DV (Distance Vector)
Message complexity	$O(nE)$ (flood LSAs)	Neighbor exchanges only
Speed of convergence	Dijkstra $O(n^2)$; may oscillate	Varies; loops, count-to-infinity
Robustness	Wrong *link* cost affects own table	Wrong *path* cost propagates; blackholes

Traffic Engineering

- Goal: optimize network performance by tuning routing (often via link weights).
- Workflow: measure traffic/topology, build network-wide model, then optimize weights.
- Common objectives: maximize throughput, minimize congestion (max-link load), fairness.
  - Minimize Congestion - minimize the maximum link utilization: $\min \max_{e \in E} \frac{\text{load}(e)}{\text{capacity}(e)}$.
  - Maximize Throughput - maximize the total accepted traffic: $\max \sum_{(s,t)} \text{flow}(s, t)$.

- Maximum Multicommodity Flow (Max-MCF): maximize total flow between all source-destination pairs subject to link capacities.
- **Commodities** are tuples (s, t, d) that represent demand d from source s to target t, e.g., $(A, B, 12)$.
- **Max-MCF$_{\text{OSPF/ECMP}}$:** maximize total flow using OSPF+ECMP (flows can split only on shortest paths).
- **Max-MCF$_{\text{OPT}}$:** maximize total flow with arbitrary splitting (no shortest-path restriction).
- **MinCong-MCF$_{\text{OSPF/ECMP}}$:** minimize max congestion using OSPF+ECMP.
- **MinCong-MCF$_{\text{OPT}}$:** minimize max congestion with arbitrary splitting. *Note: In MinCong-MCF, all commodities must be sent (even if capacities are exceeded); in Max-MCF, flows are bounded by capacities.*

- **Complexity of link-weight optimization:**
  - NP-hard, even for simple objectives.
  - **Theorem:** Approximating the min-congestion multi-commodity flow within any constant factor is NP-hard, even for a single source-target pair.

Modelling networks as linear programs

TODO ?

ECMP

- Equal-Cost Multi-Path: split traffic across multiple shortest paths (same total cost).
- Each router keeps a set of equal-cost next hops and load-balances among them.
- **Hashing**:
  - choose next hop by hashing the flow 5-tuple (src/dst IP, src/dst port, protocol).
  - This way we achieve deterministic per-flow mapping: all packets of a flow take the same path (avoids reordering).
  - Efficient to compute; provides near-even split across equal-cost paths in aggregate.
  - Caveat: if only a few "heavy" flows exist, they may map to same path $\Rightarrow$ imbalance.
- Good for load sharing, but optimality is not guaranteed (NP-hard).

Transport Layer (TCP & UDP)

TCP (Transmission Control Protocol)

- Reliable, in-order byte stream; receiver ACKs received data.
- **vs. UDP:** UDP provides port+IP multiplexing (best effort) with no reliability, ordering, or flow control.
- **Sequence #:** The byte number of the *first* byte of data in the segment.
- **ACK #:** The *next expected* byte (Cumulative ACK).
- **MTU vs. MSS:**
  - **MTU:** Max Transfer Unit (max packet size on link, e.g., 1500B).
  - **MSS:** Max Segment Size (max payload). $MSS = MTU - (IP_{Header} + TCP_{Header})$. Typically $MSS = MTU - 40$ (so $MSS < MTU$).

Connection Setup (3-Way Handshake)

1. Client $\rightarrow$ Server: **SYN**, Seq $= x$.
   Announces client ISN (Initial Sequence Number, x) and requests connection.
2. Server $\rightarrow$ Client: **SYN + ACK**, Seq $= y$, ACK $= x + 1$.
   Accepts, advertises server ISN, and ACKs client ISN.
3. Client $\rightarrow$ Server: **ACK**, Seq $= x + 1$, ACK $= y + 1$.
   Final ACK; both sides synchronized.
4. (Connection Established)

Connection Teardown

1. Client sends **FIN**.
2. Server replies **ACK**. (Client enters FIN_WAIT_2).
3. Server sends **FIN**.
4. Client replies **ACK**, waits **TIME_WAIT** (usually 2*MSL), then closes.

Reliable data transfer (sender)

- Maintain sequence number and buffer; segment and send data.
- Send while window allows: $lastByteSent - lastByteAcked \leq RW$ (receiver window).
- Start timer if not running; on timeout, retransmit the oldest unACKed segment.

Reliable data transfer (receiver)

- Deliver in-order data and send cumulative ACK.
- If out-of-order segment arrives, send ACK for last in-order byte (duplicate ACK).

Retransmissions

- Retransmit on either **timeout** or **duplicate ACKs**.
- **Fast retransmit:** many duplicate ACKs imply loss before timeout.

RTT estimation

- Goal: set timeout slightly above RTT to avoid premature retransmits but still react fast to loss.
- **SampleRTT:** time from sending a segment until its first ACK returns (ignore retransmitted samples).
- Now we can compute a smoothed RTT which is exponentially weighted moving average of recent samples:
  $$\textbf{EstimatedRTT} \leftarrow (1 - \alpha)\,\text{EstimatedRTT} + \alpha\,\text{SampleRTT}$$
  typical value for α is $1/8$ (higher α reacts faster; lower is smoother).
- RTT variation (we add some safety margin):
  $$\textbf{DevRTT} \leftarrow (1 - \beta)\,\text{DevRTT} + \beta\,|\text{SampleRTT} - \text{EstimatedRTT}|$$
  typical value for β is $1/4$ (higher β reacts faster; lower is smoother).
- We may then set the timeout interval as:
  $$\textbf{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

Flow Control vs. Congestion Control

- **Flow control**: prevents sender from overwhelming the receiver (receiver buffer limits).
- **Congestion control**: prevents sender from overloading the network.
- Sender can send while: $lastByteSent - lastByteAcked \leq \min(RW, cwnd)$.
- When receiver buffer is the limiter, flow control dominates; when network is the limiter, congestion control dominates.

Congestion Control Phases (AIMD)

AIMD (Additive Increase, Multiplicative Decrease): Ensures stability and fairness.

1. **Slow Start (SS):** Exponential growth; $cwnd = cwnd + 1$ per ACK (doubles every RTT).
2. **Congestion Avoidance (CA):** Linear growth; $cwnd = cwnd + 1/cwnd$ per ACK ($\approx +1$ MSS per RTT).

- **Transition:** Threshold **ssthresh** decides SS vs CA: If $cwnd < ssthresh \rightarrow$ SS. If $cwnd \geq ssthresh \rightarrow$ CA.

TCP Tahoe

- On **any** loss (Timeout OR 3-Dup ACKs):
- Set $ssthresh = \lfloor cwnd/2 \rfloor$.
- Set $cwnd = 1$.
- Enter **Slow Start**.

TCP Reno (Standard)

- Improves Tahoe for 3 duplicate ACKs (fast retransmit/fast recovery).
- On **Timeout:** Same as Tahoe ($cwnd = 1$, restart SS etc).
- On **3 Duplicate ACKs (Fast Retransmit/Recovery):**
  1. Retransmit missing segment immediately.
  2. Set $ssthresh = \lfloor cwnd/2 \rfloor$.
  3. Set $cwnd = ssthresh + 3$ (accounts for 3 packets buffering).
  4. Enter **Congestion Avoidance** (skips Slow Start).
- **Utilization intuition**
  - Reno provides sawtooth behavior in $cwnd$ over time.
  - With window size W and RTT RTT, throughput is about W/RTT.
  - If window halves after loss, average throughput over a cycle is roughly $0.75\,W/RTT$.

TCP Vegas

- Delay-based congestion control: compares expected throughput (from $cwnd/baseRTT$) to actual throughput (from $cwnd/RTT$).
- Adjusts $cwnd$ to keep the difference within a target range (α, β), aiming to prevent queue buildup before loss.
- Less widely used because it competes poorly with loss-based flows (Reno), is sensitive to RTT measurement noise, and offers limited incremental deployability in the public Internet.

UDP (User Datagram Protocol)

- Connectionless; no setup, no reliability, no ordering.
- Uses IP+port for multiplexing/demultiplexing (socket identified by destination port).
- Very small header; no congestion control, so can send as fast as the application wants.
- Useful when timeliness matters more than reliability (e.g., streaming, VoIP), or when simplicity/overhead is key.
- Also used for request/response protocols like DNS and for DHCP (no TCP connection possible before address assignment).
- Reliability (if needed) is handled at the application layer.

Reliable Data Transfer Protocols

These protocols are the conceptual building blocks for reliability. While TCP is a specific protocol, it uses the mechanisms defined in these models.

Stop-and-Wait

- **Mechanism:** Sender sends one packet, waits for ACK; if ACK/packet lost, timeout triggers retransmission.
- **Efficiency:** Very low in "long fat pipes" (high RTT).
- **Utilization:** $U_{sender} = \frac{L/R}{RTT + L/R}$
  (where L=packet length, R=transmission rate).
- **Timeout lower bound:** $T_{out} \geq 2T_{prop} + T_{ack} + T_{pt}$.
- $T_{prop} = \frac{distance}{propagation\ speed}$, $T_{ack} = \frac{ACK\ size}{rate}$, T_{pt} = processing time.
- **Goodput (loss prob. p for both data and ACK):**
  $$\text{Goodput} = \frac{T_{packet}}{\frac{1}{(1-p)^2}(T_{packet} + T_{out})} = \frac{T_{packet}(1-p)^2}{T_{packet} + T_{out}}.$$
  Note: Here we assume independent losses for data and ACK packets. If ACK losses are negligible, replace $(1-p)^2$ with $(1-p)$ in the Goodput formula, since now it's Geometric with success prob. $(1-p)$.
  also, if $\overline{T_{out}}$ of successful transmissions is differenet than that of retransmissions, we would need to weight accordingly.
  $$\text{Goodput} = \frac{T_{packet}}{\left(\frac{1}{(1-p)^2} - 1\right)(T_{packet} + T_{out}) + (T_{packet} + \overline{T_{out}})}$$

Pipelined Protocols (Sliding Window):
Allows multiple unacknowledged packets to be "in-flight" to increase utilization.

Go-Back-N (GBN):

- **Sender:** Can send up to N packets without ACKs. Uses a *single timer* for the oldest unACKed packet.
- **Receiver:** Uses **Cumulative ACKs**. If a packet is missing, it discards all subsequent out-of-order packets.
- **Receiver window:** size 1; accepts only next in-order packet.
- **On Timeout:** Sender retransmits *all* N transmitted but unACKed packets.
- **Goodput (loss prob. p):** let $a = \frac{T_{out} + T_{packet}}{T_{packet}}$, then
  $\text{Goodput}_{GBN} = \frac{1-p}{1-p+pa}$.

Selective Repeat (SR):

- **Sender:** Maintains a *timer for each* individual packet.
- **Receiver:** ACKs packets individually. **Buffers** out-of-order packets for later delivery.
- **On Timeout:** Sender retransmits *only* the specific lost packet.
- **Window Size Constraint:** To avoid sequence ambiguity, $WindowSize \leq \frac{1}{2} \cdot MaxSeqNum$.

Criteria	Selective Repeat	Go-Back-N
Bandwidth utilization	Retransmits only lost packets	May retransmit already received packets
Window sizes	Sender: N, Receiver: N	Sender: N, Receiver: 1
Sequence numbers	$2N$	$N + 1$
Out-of-order handling	Buffers and tracks out-of-order packets	No buffering; accepts only in-order
Receiver storage	Stores out-of-order until missing arrives	No need to store (sender may resend)

Implementation & Relation to TCP

- **Where they run:** These protocols run in the **Transport Layer** (Layer 4) within the **Operating System** of the end-hosts. Routers and switches (the "network core") generally do not implement these.
- **TCP Implementation:**
  - TCP is a hybrid: It uses **Cumulative ACKs** (like GBN) but typically **Buffers out-of-order segments** and can use **SACK** (Selective ACK) to retransmit only missing data (like SR).

Application Layer (HTTP, DNS, SMTP & FTP)

DNS (Domain Name System)

Maps between hostnames and IP addresses. Each server is authoritative for its domain and can resolve queries or forward them; resolution ends when an authoritative server replies or the requested data is found in cache.

Record types:

- **A**: maps an Internet hostname to an IP address.
- **NS**: maps a domain name (e.g., `google.com`) to the server responsible for it.
- **CNAME**: maps a hostname to its canonical name, e.g., `wedsac.mit.com` is actually `www.mit.com`.
- **PTR**: reverse of A (IP address to hostname).
- **MX**: maps a hostname to the mail server that handles it.

DNS queries and replies use UDP and typically port 53 (smaller packets for requests/replies).
Servers return a **TTL**; the answer is cached for that duration.
Glued response: if the client needs to query the authoritative server but does not know its IP, the server returns the IP in the NS response.
Security issues:

1. **DDoS Attack**: flooding DNS servers so they cannot answer other requests.
2. **MITM (Man-in-the-Middle) Attack**: when a client queries a server, a fake response can arrive before the real one, potentially mapping a requested name to a desired IP.
3. **Cache poisoning**: if the server is not well-protected, it continues serving an injected response until TTL expires; thus, we must ensure the reply is authentic.
4. **Kaminsky attack**: attacker sends many requests for random subdomains of a target domain (e.g., `random1.target.com`, `random2.target.com`, etc.) to a resolver; when the resolver queries the authoritative server, the attacker floods it with fake replies, hoping one matches the query ID and port, poisoning the cache.
- **DNSSEC**: adds signatures so you can verify who answered and add more security; adds extra information and expands UDP packet size.

DHCP (Dynamic Host Configuration Protocol)

- Allows host to obtain IP dynamically (no manual config).
- Each time host connects, it can get a different IP; server can **lease** addresses and reuse later.
- Used by devices joining a network.
- **Message flow (DORA):**
  - **DHCP Discover:** src IP 0.0.0.0, dst 255.255.255.255 (broadcast, server unknown).
  - **DHCP Offer:** server replies, dst 255.255.255.255 (broadcast); includes **yiaddr** (your IP).
  - **DHCP Request:** client broadcasts 0.0.0.0 $\rightarrow$ 255.255.255.255 to accept one offer; includes chosen **yiaddr** so other servers withdraw.
  - **DHCP ACK:** server confirms lease; may broadcast so others know address in use.
- Each DHCP message includes a unique **Transaction ID (XID)** generated by the client; even if replies are broadcast, only the matching client accepts them.
- **DHCP relaying:** if DHCP server not on local subnet, relay forwards messages to server on another network.
  - Relay agent is usually the local router; client $\leftrightarrow$ relay uses broadcast, relay $\leftrightarrow$ server uses unicast.
  - Relay inserts **giaddr** (gateway IP) so server knows the client subnet and selects an address accordingly.
  - Relay forwards Discover/Request and returns Offer/ACK back to the client (continue with DORA).
- **Example (relayed Discover/Offer steps):**
  - Setup: H1 on LAN-A, router R0 (relay agent) connects LAN-A to LAN-B; DHCP server is on LAN-B. Example IPs: R0(LAN-A)=223.1.1.3, R0(LAN-B)=223.1.2.2.

Message	SRC MAC	DST MAC	SRC IP	DST IP
DHCP Discover	MAC(H1)	Broadcast	0.0.0.0	255.255.255.255
Discover (giaddr=R0)	MAC(R0)	MAC(DHCP)	IP(R0)	IP(DHCP)
DHCP Offer (223.1.1.x)	MAC(DHCP)	MAC(R0)	IP(DHCP)	IP(R0)
DHCP Offer (223.1.1.x)	MAC(R0)	Broadcast	IP(R0)	255.255.255.255

  - After the offer, the protocol continues similarly with Request and ACK.
- **Why it doesn't fully solve IPv4 shortage:**
  - **Peak demand**: works only if simultaneously active devices $<$ pool size; if everyone connects at once, the pool still runs out.
  - **Temporary fix**: improves reuse but does not increase the global IPv4 address space (2^{32}).

Interdomain Routing & BGP

Autonomous Systems (AS)

- **Definition:** IP networks/routers under one admin with a common routing policy.
- **ASN:** Unique 16/32-bit AS identifier used in BGP.
- **AS relationships:**
  - **Customer / Provider:** Customer pays provider for transit.
  - **Full / Partial transit:** Full = global reachability; partial = limited reach.
  - **Peering:** Settlement-free; advertise own + customers only.
- **AS-level topology:** Common categories
  - **Tier-1:** No providers; global reach via peering.
  - **Tier-2:** Providers + peers (buys some transit).
  - **Stub AS:** Single provider; no downstream customers.

Border Gateway Protocol (BGP)

- **Definition:** Inter-AS routing protocol.
- **Path-vector:** Advertises AS_PATH; reject if own ASN appears.
- **Policy + attributes:** LOCAL_PREF, MED, AS_PATH, NEXT_HOP, etc.

BGP message flow (5 stages)

1. **TCP setup:** Session on TCP/179.
2. **OPEN:** Exchange ASN, hold-time, capabilities.
3. **Initial UPDATE:** Send NLRI + attributes.
4. **Decision:** Import policy + best-path selection.
5. **Steady state:** KEEPALIVE + incremental UPDATE/NOTIFICATION.

iBGP, eBGP and IGP interaction

- **eBGP:** Between ASes; increments AS_PATH, may change NEXT_HOP.
- **iBGP:** Within AS; no AS_PATH prepend; needs full mesh or route-reflectors.
- **IGP:** Provides internal reachability to BGP NEXT_HOP.

BGP Stability

- **Stable state:** Routes converge with no further changes.
- **Safety theorem (informal):** Under Gao-Rexford, BGP converges to a stable outcome.
- **Gao-Rexford conditions (commercial routing)**
  - **Topology:** Customer-provider edges form a DAG; peers are lateral.
  - **Preference:** Customer $>$ peer $>$ provider.
  - **Export:** Export all routes to customers; only customer routes to peers/providers.